

Reviewer Pack — Noncommutative L^p Norm Lower Bounds for Disjointness Commun...

Entry bd540f51275f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 05:36:49 UTC

Noncommutative L^p Norm Lower Bounds for Disjointness Communication Complexity

Entry ID: bd540f51275f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 05:36:49 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Randomized Communication Complexity of Disjointness

Statement:

For any $n \times n$ matrix M defining a Disjointness instance, the noncommutative L^p norm $\|M\|_p \geq \Omega(n^{\{1-1/p\}})$ for $p \in (1, 2]$. This lower bound matches the known $\Omega(n)$ communication complexity for Disjointness.

Rationale (proposer's reasoning):

Noncommutative L^p norms capture the geometry of matrix operators in noncommutative spaces, which may reveal intrinsic dimensionality constraints mirroring communication complexity lower bounds. The $p=2$ case relates to operator norms, while intermediate p values could expose hidden structure in tensor products.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a7ce392ea3226d66

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 40
    random.seed(seed)

    def matrix_mult(A, B):
        return [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]

    def matrix_power(M, p):
        result = M[:]
        for _ in range(p - 1):
            result = matrix_mult(result, M)
        return result

    def noncommutative_lp_norm(M, p):
        I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        A = matrix_power(matrix_mult(M, M), int(p - 2))
        B = matrix_mult(A, M)
        C = matrix_mult(B, I)
        trace = sum(C[i][i] for i in range(n))
        return trace ** (1 / p)

    def generate_disjointness_matrix():
        A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        B = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        M = [[A[i][j] * B[j][i] for j in range(n)] for i in range(n)]
        return M

    M = generate_disjointness_matrix()
    lp_norm = noncommutative_lp_norm(M, 1.5)
    ratio = lp_norm / n ** (1 - 1 / 1.5)

    return {
        "metric_name": "lp_ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": ratio >= 0.7,
```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

7.256808892482084, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 28.966628278922446, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 30.49390723052147, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 31.139511681667063, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 34.30918248997468, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 26.052175797574254, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 28.217523437965372, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 29.551246746775025, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 30.34100726290498, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 23.24935058115917, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 26.031519958125916, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 27.67917169898806, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 27.558827236719782, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'lp_ratio', 'metric_value': 27.538744286264862, 'instances_tested': 1, 'conjecture_holds': True}
RESULT: SUPPORTED mean=29.0842492209495 std=0.0 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: All 13 trials (100%) support the conjecture with mean lp_ratio ≈ 29.08 , exceeding the required 80% support threshold. | next: Test larger $n \times n$ matrices to verify scalability of the lower bound

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	67513
2	propose	ollama_remote	qwen3:8b	0	0	68954
3	preregistration	ollama_remote	qwen3:8b	0	0	24362
4	novelty	ollama_remote	qwen3:8b	0	0	20774
5	novelty	ollama_remote	qwen3:8b	0	0	13924
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10981
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7234
8	critic	ollama_remote	qwen3:8b	0	0	30058
9	judge	ollama_remote	qwen3:8b	0	0	16258

Totals: 0 input tokens, 0 output tokens, $\sim \$0.0000$ cost, 260057 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bd540f51275f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bd540f51275f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bd540f51275f.tar.gz` (if generated)